MODULE 8

Pandas

Topics Covered

Lecture 1 — Pandas Introduction
Lecture 2 — Getting Started with Series
Lecture 3 — Mathematical Functions & Indexing in Series
Lecture 4 — Real World Datasets in Series
Lecture 5 — DataFrame Introduction
Lecture 6 — Python Functionalities in Series & Plotting Graphs
Lecture 7 — Creating DataFrames, Attributes & Functions
Lecture 8 — Fetching Specific Records and Columns
Lecture 9 — Filtering, Adding Columns & Other Important Functions

Lecture 1: Pandas Introduction

Pandas is Python's go-to library for data manipulation and analysis. Built on top of NumPy, it introduces two key structures — Series and DataFrame — that make working with tabular and time-series data intuitive and fast.

> **Why Pandas?**
> Excel has rows & columns — so does a Pandas DataFrame.
> But Pandas handles millions of rows in milliseconds, supports missing data natively, integrates with ML libraries, and runs in code (reproducible & automatable).

Installation & Import

```
pip install pandas

import pandas as pd      # 'pd' is the universal alias
import numpy as np
print(pd.__version__)
```

Two Core Data Structures

Structure	Description | Think of it as...
Series	1D labelled array → A single column of a spreadsheet
DataFrame	2D labelled table → A full spreadsheet with rows & columns

> **Pandas vs NumPy**
> NumPy: fast numerical computing on homogeneous arrays (all same dtype).
> Pandas: labelled, heterogeneous tabular data (columns can have different dtypes) + built-in data cleaning tools.

Lecture 2: Getting Started with Series

A Series is a one-dimensional labelled array. Every element has an index (label) — by default 0, 1, 2… but you can set custom labels.

Creating a Series

```
import pandas as pd

# From list (default integer index)
s1 = pd.Series([10, 20, 30, 40, 50])

# From list with custom index
s2 = pd.Series([85, 92, 78], index=['Maths', 'Science', 'English'])

# From dictionary (keys become index)
```

```
s3 = pd.Series({'Mumbai': 20.7, 'Delhi': 31.1, 'Bangalore': 17.5})

# From scalar (broadcasts to all index positions)
s4 = pd.Series(5, index=['a', 'b', 'c', 'd'])

print(s2['Maths'])     # 85
print(s2[0])           # 85  (positional also works)
```

Series Attributes

Attribute	Description
s.values	NumPy array of the values
s.index	Index object (labels)
s.dtype	Data type of values
s.shape	Shape tuple, e.g. (5,)
s.size	Total number of elements
s.name	Name of the Series

```
s = pd.Series([10, 20, 30], index=['a','b','c'], name='scores')
print(s.values)   # [10 20 30]
print(s.index)    # Index(['a', 'b', 'c'], dtype='object')
print(s.name)     # scores
```

Accessing Elements

```
s = pd.Series([100, 200, 300, 400], index=['w','x','y','z'])

print(s['x'])          # 200  — by label
print(s[1])            # 200  — by position
print(s[['w','z']])    # multiple labels
print(s['x':'z'])      # slicing by label (inclusive!)
print(s[1:3])          # slicing by position (exclusive end)
```

Label vs position slicing

s['a':'c'] — label-based: INCLUSIVE of end label.
s[0:3] — position-based: EXCLUSIVE of end index.
Same as Python lists for position, but different for labels!

Lecture 3: Mathematical Functions & Indexing in Series

Arithmetic on Series

Arithmetic aligns on index labels automatically. Where labels don't match, the result is NaN (Not a Number):

```
s1 = pd.Series([1, 2, 3], index=['a','b','c'])
s2 = pd.Series([10, 20, 40], index=['a','b','d'])

print(s1 + s2)
```

```
# a    11.0
# b    22.0
# c     NaN   ← 'c' not in s2
# d     NaN   ← 'd' not in s1

# fill_value replaces NaN during operation
print(s1.add(s2, fill_value=0))
# a    11.0   b    22.0   c    3.0   d    40.0
```

Common Mathematical Methods

Method	Description
s.sum()	Sum of all values
s.mean()	Arithmetic mean
s.median()	Median
s.std()	Standard deviation
s.var()	Variance
s.min() / s.max()	Minimum / Maximum
s.abs()	Absolute values
s.cumsum()	Cumulative sum
s.round(n)	Round to n decimal places
s.describe()	Summary stats: count, mean, std, min, quartiles, max

```
s = pd.Series([4, 7, 2, 9, 1, 5])
print(s.describe())
# count    6.000
# mean     4.667
# std      2.944
# min      1.000
# 25%      2.750
# 50%      4.500
# 75%      6.500
# max      9.000
```

loc vs iloc

Indexer	Type | Behaviour
.loc[ ]	Label-based → uses index labels; end is INCLUSIVE
.iloc[ ]	Position-based → uses integer positions; end is EXCLUSIVE

```
s = pd.Series([10,20,30,40,50], index=['a','b','c','d','e'])

print(s.loc['b':'d'])    # b=20, c=30, d=40  (inclusive)
print(s.iloc[1:4])       # pos 1,2,3 → 20, 30, 40 (exclusive end)
print(s.loc[['a','c']])  # 10, 30  — multiple labels
print(s.iloc[[0,2,4]])   # 10, 30, 50 — multiple positions
```

Lecture 4: Real World Datasets in Series

Reading Data into a Series

```
import pandas as pd

# Read a CSV column directly as Series
df = pd.read_csv('data.csv')
prices = df['Price']           # one column = Series

# Read a single-column CSV as Series
s = pd.read_csv('prices.csv', squeeze=True)
```

Handling Missing Data (NaN)

Real datasets almost always have missing values. Pandas represents them as NaN (float) or pd.NaT (for dates).

```
s = pd.Series([10, None, 30, None, 50])

print(s.isnull())          # True where NaN
print(s.notnull())         # True where NOT NaN
print(s.isnull().sum())    # count of missing values: 2

# Drop NaN
print(s.dropna())

# Fill NaN with a value
print(s.fillna(0))                  # replace with 0
print(s.fillna(s.mean()))           # replace with mean
print(s.fillna(method='ffill'))     # forward fill
print(s.fillna(method='bfill'))     # backward fill
```

Useful Exploration Methods

Method	Description
s.head(n)	First n elements (default 5)
s.tail(n)	Last n elements (default 5)
s.value_counts()	Frequency of each unique value
s.unique()	Array of unique values
s.nunique()	Count of unique values
s.sort_values()	Sort by value
s.sort_index()	Sort by index
s.reset_index()	Reset index to 0,1,2...

```
s = pd.Series(['cat','dog','cat','bird','dog','dog'])
print(s.value_counts())
# dog     3
# cat     2
# bird    1

print(s.unique())      # ['cat' 'dog' 'bird']
print(s.nunique())     # 3
```

Lecture 5: DataFrame Introduction

A DataFrame is a 2D labelled data structure with rows and columns — like a spreadsheet or SQL table. Each column is a Series, all sharing the same row index.

Creating a DataFrame

```
import pandas as pd

# From dictionary (keys = column names)
data = {
    'Name':   ['Alice', 'Bob', 'Charlie', 'Diana'],
    'Age':    [25, 30, 35, 28],
    'Score':  [88.5, 92.0, 79.5, 95.0],
    'City':   ['Mumbai', 'Delhi', 'Pune', 'Chennai']
}
df = pd.DataFrame(data)

# From list of dicts
rows = [{'Name':'Alice','Age':25}, {'Name':'Bob','Age':30}]
df2 = pd.DataFrame(rows)

# From NumPy array
import numpy as np
arr = np.random.randint(1,100,(4,3))
df3 = pd.DataFrame(arr, columns=['A','B','C'])
```

Reading from Files

Function	Use Case
pd.read_csv('file.csv')	Read comma-separated file
pd.read_excel('file.xlsx')	Read Excel spreadsheet
pd.read_json('file.json')	Read JSON file
pd.read_sql(query, conn)	Read from SQL database
df.to_csv('out.csv', index=False)	Save to CSV
df.to_excel('out.xlsx')	Save to Excel

```
df = pd.read_csv('employees.csv')

# Common parameters
pd.read_csv('data.csv',
    sep=',',            # delimiter
    header=0,           # row to use as column names
    index_col='ID',     # set a column as index
    usecols=['Name','Salary'],  # read only these columns
    nrows=100,          # read only first 100 rows
    na_values=['NA','?','--'],  # treat these as NaN
)
```

Lecture 6: Python Functionalities in Series & Plotting Graphs

Applying Python Functions to Series

```
s = pd.Series([1, 4, 9, 16, 25])

# apply() — apply any function element-wise
print(s.apply(lambda x: x ** 0.5))   # square root each

import math
print(s.apply(math.sqrt))

# map() — map values to new values
grades = pd.Series(['A','B','A','C','B'])
grade_map = {'A': 4.0, 'B': 3.0, 'C': 2.0}
print(grades.map(grade_map))   # [4.0, 3.0, 4.0, 2.0, 3.0]

# String operations via .str accessor
names = pd.Series(['alice', 'bob', 'charlie'])
print(names.str.upper())          # ['ALICE','BOB','CHARLIE']
print(names.str.len())            # [5, 3, 7]
print(names.str.contains('a'))    # [True, False, True]
print(names.str.replace('a','@')) # ['@lice', 'bob', 'ch@rlie']
```

Plotting with Pandas

Pandas wraps Matplotlib to allow quick plots directly from Series and DataFrames. Always import matplotlib.pyplot to display plots.

```
import matplotlib.pyplot as plt

s = pd.Series([3, 7, 2, 9, 5, 8], index=['Jan','Feb','Mar','Apr','May','Jun'])

s.plot(kind='line', title='Monthly Sales', color='blue')
plt.show()

s.plot(kind='bar',  color='steelblue')
plt.show()

s.plot(kind='hist', bins=5)
plt.show()

s.plot(kind='pie', autopct='%1.1f%%')
plt.show()
```

Plot Kind	Best For
'line'	Trends over time / ordered categories
'bar'	Comparing categories
'barh'	Horizontal bar — long category names
'hist'	Distribution of numerical data
'box'	Spread, median, outliers
'pie'	Proportions (use sparingly)
'scatter'	Relationship between two variables (DataFrame)

Lecture 7: Creating DataFrames, Attributes & Functions

Key DataFrame Attributes

Attribute	Description | Example Output
df.shape	(rows, cols) → (4, 3)
df.dtypes	dtype of each column
df.columns	Index of column names
df.index	Row index (RangeIndex or custom)
df.size	Total elements = rows × cols
df.ndim	Always 2 for DataFrame
df.info()	Summary: col names, dtypes, non-null counts, memory
df.describe()	Stats for numeric columns: mean, std, min, quartiles, max

```
df = pd.read_csv('employees.csv')

print(df.shape)        # (200, 6)
print(df.dtypes)
# Name       object
# Age         int64
# Salary    float64

df.info()
# <class 'pandas.core.frame.DataFrame'>
# RangeIndex: 200 entries, 0 to 199
# Data columns (total 6 columns):
#  #   Column    Non-Null Count  Dtype
# ...

print(df.describe())
```

Viewing Data

```
df.head(5)         # first 5 rows (default)
df.tail(5)         # last 5 rows
df.sample(5)       # 5 random rows
df.sample(frac=0.1) # random 10% of rows
```

Renaming & Setting Index

```
# Rename columns
df.rename(columns={'Name': 'FullName', 'Sal': 'Salary'}, inplace=True)

# Set a column as the row index
df.set_index('EmployeeID', inplace=True)

# Reset to default integer index
df.reset_index(inplace=True)
```

Handling Missing Values in DataFrame

```
df.isnull().sum()            # missing count per column
df.isnull().sum().sum()      # total missing values

df.dropna()                  # drop rows with ANY NaN
df.dropna(axis=1)            # drop columns with ANY NaN
df.dropna(thresh=3)          # keep rows with at least 3 non-NaN

df.fillna(0)                 # fill all NaN with 0
df['Age'].fillna(df['Age'].mean(), inplace=True)  # fill with mean
df.fillna(method='ffill')    # forward fill
```

Lecture 8: Important Functions & Fetching Specific Records and Columns

Selecting Columns

```
# Single column → returns Series
df['Name']
df.Name           # dot notation (only if name has no spaces)

# Multiple columns → returns DataFrame
df[['Name', 'Salary', 'Department']]
```

Selecting Rows — loc & iloc

```
# loc — label-based (uses index labels)
df.loc[0]                    # row with index label 0
df.loc[0:3]                  # rows 0,1,2,3 (INCLUSIVE)
df.loc[0:3, 'Name':'Age']    # rows 0-3, columns Name to Age
df.loc[[2,5,8], ['Name','Salary']]  # specific rows & cols

# iloc — position-based (integer positions)
df.iloc[0]                   # first row
df.iloc[0:3]                 # rows 0,1,2 (EXCLUSIVE end)
df.iloc[0:3, 0:2]            # first 3 rows, first 2 cols
df.iloc[-1]                  # last row
df.iloc[[0,2,4], [1,3]]      # specific row & col positions
```

loc vs iloc — key difference

df.loc[0:5] → 6 rows (0,1,2,3,4,5 — label end is INCLUSIVE)
df.iloc[0:5] → 5 rows (0,1,2,3,4 — position end is EXCLUSIVE)

Sorting Data

```
df.sort_values('Salary')                          # ascending
df.sort_values('Salary', ascending=False)         # descending
df.sort_values(['Dept', 'Salary'], ascending=[True, False])
# sort by Dept asc, then Salary desc

df.sort_index()         # sort by row index
```

Applying Functions to DataFrames

```
# apply() on a column (Series)
df['Salary_K'] = df['Salary'].apply(lambda x: x / 1000)

# apply() on entire DataFrame row/col
df[['A','B','C']].apply(np.sum, axis=0)  # col sums
df[['A','B','C']].apply(np.sum, axis=1)  # row sums

# applymap() — element-wise on whole DataFrame
df.applymap(lambda x: round(x, 2))       # round all values

# map() — on a single Series
df['Grade'] = df['Score'].map({90:'A', 80:'B', 70:'C'})
```

Lecture 9: Filtering, Adding New Columns & Other Important Functions

Filtering Rows (Boolean Indexing)

```
# Single condition
df[df['Age'] > 30]
df[df['City'] == 'Mumbai']

# Multiple conditions — use & (and), | (or), ~ (not)
df[(df['Age'] > 25) & (df['Salary'] > 50000)]
df[(df['City'] == 'Mumbai') | (df['City'] == 'Delhi')]
df[~df['Name'].str.contains('a')]   # names NOT containing 'a'

# isin() — match against a list of values
df[df['City'].isin(['Mumbai', 'Pune', 'Nagpur'])]

# between() — range filter
df[df['Age'].between(25, 35)]

# query() — SQL-like string syntax
df.query('Age > 30 and Salary > 50000')
```

Adding & Modifying Columns

```
# Add a new column
df['Tax'] = df['Salary'] * 0.2
df['Net_Salary'] = df['Salary'] - df['Tax']

# Conditional column with np.where
import numpy as np
df['Category'] = np.where(df['Score'] >= 90, 'Excellent', 'Good')

# Multiple conditions with np.select
conditions  = [df['Score']>=90, df['Score']>=75, df['Score']>=60]
choices     = ['A', 'B', 'C']
df['Grade'] = np.select(conditions, choices, default='F')
```

```
# Drop a column
df.drop(columns=['Tax'], inplace=True)

# Rename a column
df.rename(columns={'Net_Salary': 'Take_Home'}, inplace=True)
```

groupby — Split, Apply, Combine

groupby splits data into groups, applies a function to each group, and combines the results — the most powerful analysis tool in Pandas.

```
# Average salary by department
df.groupby('Department')['Salary'].mean()

# Multiple aggregations at once
df.groupby('Department').agg({
    'Salary': ['mean', 'max', 'min'],
    'Age':    'mean',
    'Name':   'count'
})

# Group by multiple columns
df.groupby(['Department', 'City'])['Salary'].sum()

# size() — count rows in each group
df.groupby('City').size()
```

Merging & Joining DataFrames

```
# merge — like SQL JOIN
result = pd.merge(df1, df2, on='EmployeeID', how='inner')
# how: 'inner' | 'left' | 'right' | 'outer'

# concat — stack vertically or horizontally
combined = pd.concat([df1, df2])              # vertical (add rows)
combined = pd.concat([df1, df2], axis=1)      # horizontal (add cols)
```

Other Important Functions

Function	Description
df.duplicated()	Boolean mask of duplicate rows
df.drop_duplicates()	Remove duplicate rows
df.replace(old, new)	Replace specific values
df['col'].astype(dtype)	Change column data type
df.pivot_table()	Excel-style pivot table
df.melt()	Unpivot: wide format → long format
df.corr()	Correlation matrix between numeric columns
df.value_counts()	Frequency count for each unique combo
df.T	Transpose the DataFrame

Function	Description

```
# Correlation matrix
print(df[['Age','Salary','Score']].corr())
# Values close to 1 = strong positive correlation
# Values close to -1 = strong negative correlation
# Values close to 0 = no correlation

# Pivot table
df.pivot_table(values='Salary', index='Department',
               columns='City', aggfunc='mean')
```

End of Module 8 Notes — Generative AI Course